

Rapport de travaux pratiques en C++

Piles génériques et expressions mathématiques

Réalisé par : **Hamri Meriem**

Année universitaire 2026–2027

1 Problématique

Le but de ce travail est de créer une pile générique de deux manières, avec un tableau dynamique et avec une liste chaînée. Ces piles serviront ensuite à vérifier une expression mathématique et à convertir une expression infixe en expression préfixée.

Une expression comme $\{(x+y)+z\}*5$ est incorrecte : l'accolade ouvrante est suivie d'une parenthèse fermante qui ne lui correspond pas. Il faut aussi vérifier que les opérandes et les opérateurs apparaissent dans un ordre cohérent, puis respecter les priorités lors de la conversion.

2 Méthodologie générale

Le programme procède en trois étapes. Il découpe d'abord le texte saisi en jetons (nombres, variables, opérateurs et délimiteurs). Il vérifie ensuite la syntaxe et l'ordre des délimiteurs avec une pile. Enfin, si l'expression est valide, il emploie deux piles pour former l'écriture préfixée.

Les opérateurs pris en charge sont $+$, $-$, $*$, $/$ et $^$. Le programme reconnaît les groupes $()$, $[]$ et $\{\}$. Les opérateurs sont binaires : un signe unaire comme $-x$, les fonctions et la multiplication implicite ne sont pas traités.

3 Question 1 : piles génériques

Le mot-clé `template` permet d'utiliser une pile avec différents types, par exemple `char`, `std::string` ou une paire contenant un caractère et sa position. Les deux classes proposent les mêmes opérations : `empiler`, `depiler`, `sommet`, `vide` et `taille`. Une lecture ou un retrait sur une pile vide provoque une exception.

Le début du fichier `piles.hpp` contient les inclusions nécessaires aux deux classes.

```
#pragma once
```

```
#include <cstddef>
#include <memory>
#include <stdexcept>
#include <utility>
```

```
// Meme interface pour les deux realisations : empiler, depiler, sommet, vide, taille.
// Les copies sont interdites afin de garder la gestion memoire lisible dans ce TP.
```

3.1 Pile avec tableau dynamique

Dans `PileTableau<T>`, les valeurs sont placées dans une zone contiguë. Lorsque la capacité est atteinte, `agrandir()` alloue une zone deux fois plus grande et y déplace les éléments. `empiler()` ajoute en fin de tableau; `depiler()` retire le dernier élément. Le destructeur libère la mémoire réservée. La copie de la pile est désactivée pour éviter de dupliquer la zone mémoire sans une gestion explicite.

```

template <typename T>
class PileTableau {
    std::allocator<T> alloc_;
    T* donnees_ = nullptr;
    std::size_t taille_ = 0;
    std::size_t capacite_ = 0;

    void agrandir() {
        const std::size_t nouvelle = capacite_ == 0 ? 4 : capacite_ * 2;
        T* bloc = alloc_.allocate(nouvelle);
        std::size_t i = 0;
        try {
            for (; i < taille_; ++i)
                std::allocator_traits<std::allocator<T>>::construct(
                    alloc_, bloc + i, std::move_if_noexcept(donnees_[i]));
        } catch (...) {
            while (i != 0) std::allocator_traits<std::allocator<T>>::destroy(alloc_, bloc + --i);
            alloc_.deallocate(bloc, nouvelle);
            throw;
        }
        for (std::size_t j = 0; j < taille_; ++j)
            std::allocator_traits<std::allocator<T>>::destroy(alloc_, donnees_ + j);
        if (donnees_) alloc_.deallocate(donnees_, capacite_);
        donnees_ = bloc;
        capacite_ = nouvelle;
    }

public:
    PileTableau() = default;
    PileTableau(const PileTableau&) = delete;
    PileTableau& operator=(const PileTableau&) = delete;
    PileTableau(PileTableau&&) = delete;
    PileTableau& operator=(PileTableau&&) = delete;

    ~PileTableau() {
        for (std::size_t i = 0; i < taille_; ++i)
            std::allocator_traits<std::allocator<T>>::destroy(alloc_, donnees_ + i);
        if (donnees_) alloc_.deallocate(donnees_, capacite_);
    }

    bool vide() const { return taille_ == 0; }
    std::size_t taille() const { return taille_; }

    template <typename U>
    void empiler(U&& valeur) {
        if (taille_ == capacite_) agrandir();
        std::allocator_traits<std::allocator<T>>::construct(
            alloc_, donnees_ + taille_, std::forward<U>(valeur));
        ++taille_;
    }

    const T& sommet() const {
        if (vide()) throw std::out_of_range("Sommet d'une pile vide");
        return donnees_[taille_ - 1];
    }

    T depiler() {
        if (vide()) throw std::out_of_range("Depilement d'une pile vide");
        T valeur = std::move(donnees_[taille_ - 1]);
        --taille_;
        std::allocator_traits<std::allocator<T>>::destroy(alloc_, donnees_ + taille_);
        return valeur;
    }
};

```

3.2 Pile avec liste chaînée

Dans `PileListe<T>`, chaque maillon contient une valeur et un lien vers le suivant. La tête représente le sommet de la pile. `empiler()` crée une nouvelle tête et `depiler()` retire la tête actuelle. Les liens sont gérés par `std::unique_ptr`, ce qui évite les suppressions manuelles des maillons.

```
template <typename T>
class PileListe {
    struct Noeud {
        T valeur;
        std::unique_ptr<Noeud> suivant;
        template <typename U>
        Noeud(U&& v, std::unique_ptr<Noeud> s)
            : valeur(std::forward<U>(v)), suivant(std::move(s)) {}
    };
    std::unique_ptr<Noeud> tete_;
    std::size_t taille_ = 0;

public:
    PileListe() = default;
    PileListe(const PileListe&) = delete;
    PileListe& operator=(const PileListe&) = delete;
    PileListe(PileListe&&) = delete;
    PileListe& operator=(PileListe&&) = delete;

    ~PileListe() {
        // Liberation iterative : evite une longue chaine de destructeurs recursifs.
        while (tete_) tete_ = std::move(tete_->suivant);
    }

    bool vide() const { return taille_ == 0; }
    std::size_t taille() const { return taille_; }

    template <typename U>
    void empiler(U&& valeur) {
        auto nouveau = std::make_unique<Noeud>(std::forward<U>(valeur), nullptr);
        nouveau->suivant = std::move(tete_);
        tete_ = std::move(nouveau);
        ++taille_;
    }

    const T& sommet() const {
        if (vide()) throw std::out_of_range("Sommet d'une pile vide");
        return tete_->valeur;
    }

    T depiler() {
        if (vide()) throw std::out_of_range("Depilement d'une pile vide");
        T valeur = std::move(tete_->valeur);
        tete_ = std::move(tete_->suivant);
        --taille_;
        return valeur;
    }
};
```

4 Question 2 : vérifier l'expression

4.1 Découper la saisie

Les fonctions `ouvrant()`, `fermant()` et `correspondant()` reconnaissent les délimiteurs. `decouper()` parcourt l'expression pour construire une liste de jetons; chaque jeton garde son type,

son texte et sa position. Les espaces sont ignorés. Un caractère inconnu ou un nombre décimal incomplet déclenche une erreur.

```
#include "piles.hpp"

#include <cctype>
#include <iostream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

enum class Type { Operande, Operateur, Ouvrant, Fermant };
struct Jeton { Type type; std::string texte; std::size_t position; };

bool ouvrant(char c) { return c == '(' || c == '[' || c == '{'; }
bool fermant(char c) { return c == ')' || c == ']' || c == '}'; }
bool correspondant(char gauche, char droite) {
    return (gauche == '(' && droite == ')') ||
           (gauche == '[' && droite == ']') ||
           (gauche == '{' && droite == '}');
}

bool operateur(char c) { return c == '+' || c == '-' || c == '*' || c == '/' || c == '^'; }
int priorite(char c) { return c == '^' ? 3 : (c == '*' || c == '/' ? 2 : 1); }

std::vector<Jeton> decouper(const std::string& expression) {
    std::vector<Jeton> jetons;
    for (std::size_t i = 0; i < expression.size(); ) {
        unsigned char c = static_cast<unsigned char>(expression[i]);
        if (std::isspace(c)) { ++i; continue; }
        const std::size_t debut = i;
        if (std::isalpha(c) || c == '_') {
            do { ++i; } while (i < expression.size() &&
                               (std::isalnum(static_cast<unsigned char>(expression[i])) || expression[i] == '_'))
        ;
            jetons.push_back({Type::Operande, expression.substr(debut, i - debut), debut});
        } else if (std::isdigit(c)) {
            do { ++i; } while (i < expression.size() &&
                               std::isdigit(static_cast<unsigned char>(expression[i])));
            if (i < expression.size() && expression[i] == '.') {
                ++i;
                if (i == expression.size() || !std::isdigit(static_cast<unsigned char>(expression[
i]))))
                    throw std::invalid_argument("Nombre decimal incomplet a la position " + std::
to_string(debut + 1));
                do { ++i; } while (i < expression.size() &&
                                   std::isdigit(static_cast<unsigned char>(expression[i])));
            }
            jetons.push_back({Type::Operande, expression.substr(debut, i - debut), debut});
        } else if (operateur(c) || ouvrant(c) || fermant(c)) {
            Type type = operateur(c) ? Type::Operateur : (ouvrant(c) ? Type::Ouvrant : Type::
Fermant);
            jetons.push_back({type, expression.substr(i, 1), i});
            ++i;
        } else {
            throw std::invalid_argument("Caractere inconnu a la position " + std::to_string(i + 1)
);
        }
    }
    return jetons;
}
```

4.2 Vérifier l'ordre et les délimiteurs

La variable `attendOperande` indique ce qui est attendu à chaque étape. Après un opérateur, il faut un opérande ou un groupe ouvrant. Après un opérande ou un groupe fermé, il faut un opérateur ou la fin de l'expression. Chaque délimiteur ouvrant est placé dans la pile avec sa position. Lorsqu'un fermant arrive, il doit correspondre au dernier ouvrant empilé. À la fin, la pile doit être vide et l'expression ne doit pas se terminer par un opérateur.

```
template <template <typename> class Pile>
void verifier(const std::vector<Jeton>& jetons) {
    if (jetons.empty()) throw std::invalid_argument("Expression vide");
    Pile<std::pair<char, std::size_t>> delimites;
    bool attendOperande = true;
    for (const Jeton& j : jetons) {
        const char c = j.texte[0];
        const std::string position = " a la position " + std::to_string(j.position + 1);
        if (j.type == Type::Operande) {
            if (!attendOperande) throw std::invalid_argument("Operateur manquant" + position);
            attendOperande = false;
        } else if (j.type == Type::Ouvrant) {
            if (!attendOperande) throw std::invalid_argument("Operateur manquant avant le groupe"
+ position);
            delimites.empiler(std::make_pair(c, j.position));
        } else if (j.type == Type::Fermant) {
            if (attendOperande) throw std::invalid_argument("Operande manquant avant le fermant" +
position);
            if (delimites.vide() || !correspondant(delimites.sommet().first, c))
                throw std::invalid_argument("Delimites non correspondants" + position);
            delimites.depiler();
            attendOperande = false;
        } else {
            if (attendOperande) throw std::invalid_argument("Operande manquant avant l'operateur"
+ position);
            attendOperande = true;
        }
    }
    if (attendOperande) throw std::invalid_argument("Expression terminee par un operateur ou un
groupe incomplet");
    if (!delimites.vide())
        throw std::invalid_argument("Delimiteur non ferme a la position " +
std::to_string(delimites.sommet().second + 1));
}
```

Pour $\{(x+y)+z\}*5$, le dernier ouvrant encore dans la pile au second `}` est `{`. Le programme détecte donc que `{` et `}` ne correspondent pas.

5 Question 3 : conversion infix vers préfix

Une expression infix place l'opérateur entre deux opérandes : $x+y$. Une expression préfixée place l'opérateur avant eux : $+ x y$. La conversion utilise une pile d'opérateurs et une pile de sous-expressions.

La fonction `reduire()` retire l'opérateur, puis la sous-expression droite et la gauche. Elle construit la chaîne `opérateur gauche droite` et la replace dans la pile. `enPrefixe()` décide quand réduire selon la priorité des opérateurs et les délimiteurs. L'opérateur $\wedge$ est associatif à droite; les autres sont associatifs à gauche.

```
template <template <typename> class Pile>
void reduire(Pile<char>& operations, Pile<std::string>& expressions) {
    char op = operations.depiler();
    std::string droite = expressions.depiler();
```

```

    std::string gauche = expressions.depiler();
    expressions.empiler(std::string(1, op) + " " + gauche + " " + droite);
}

template <template <typename> class Pile>
std::string enPrefixe(const std::vector<Jeton>& jetons) {
    verifier<Pile>(jetons);
    Pile<char> operations;
    Pile<std::string> expressions;
    for (const Jeton& j : jetons) {
        char c = j.texte[0];
        if (j.type == Type::Operande) {
            expressions.empiler(j.texte);
        } else if (j.type == Type::Ouvrant) {
            operations.empiler(c);
        } else if (j.type == Type::Fermant) {
            while (!ouvrant(operations.sommet())) reduire(operations, expressions);
            operations.depiler();
        } else {
            // ^ est associatif a droite; les autres operateurs le sont a gauche.
            while (!operations.vide() && !ouvrant(operations.sommet()) &&
                (priorite(operations.sommet()) > priorite(c) ||
                 (priorite(operations.sommet()) == priorite(c) && c != '^')))
                reduire(operations, expressions);
            operations.empiler(c);
        }
    }
    while (!operations.vide()) reduire(operations, expressions);
    return expressions.depiler();
}

```

Le programme principal propose le choix de la pile, lit l'expression, appelle le découpage et la conversion, puis affiche soit le résultat, soit la raison du rejet.

```

int main() {
    std::cout << "Choisir la pile : 1 = tableau dynamique, 2 = liste chainee : ";
    int choix;
    if (!(std::cin >> choix) || (choix != 1 && choix != 2)) {
        std::cerr << "Choix invalide.\n";
        return 1;
    }
    std::cin.ignore(10000, '\n');
    std::cout << "Expression infixe : ";
    std::string expression;
    if (!std::getline(std::cin, expression)) return 1;
    try {
        const auto jetons = decouper(expression);
        const std::string resultat = choix == 1 ? enPrefixe<PileTableau>(jetons)
                                                : enPrefixe<PileListe>(jetons);
        std::cout << "Expression valide.\nExpression prefixe : " << resultat << '\n';
    } catch (const std::invalid_argument& erreur) {
        std::cout << "Expression invalide : " << erreur.what() << '\n';
        return 1;
    }
}

```

6 Résultats

Les cas suivants ont été exécutés avec les deux réalisations de la pile :

Expression infixe	Résultat
$\{(x+y)+z\}*5$	Invalide : délimiteurs non correspondants
$\{(x+y)+z\}*5$	$* + + x y z 5$
$a+b*c$	$+ a * b c$
$(a+b)*(c-2)$	$* + a b - c 2$
$a+*b$	Invalide : opérande manquant

Par exemple, $\{(x+y)+z\}*5$ est valide. L'addition intérieure devient $+ x y$, l'addition avec z devient $+ + x y z$, puis la multiplication donne $* + + x y z 5$.

Pour compiler le projet, placer `main.cpp` et `piles.hpp` dans le même dossier, puis exécuter :

```
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o expressions
./expressions
```

7 Conclusion

Les deux piles génériques remplissent le même rôle malgré une organisation différente en mémoire. Elles permettent de retrouver le dernier délimiteur ouvert et de traiter les opérateurs dans le bon ordre. Le programme vérifie les erreurs syntaxiques simples avant de produire l'expression préfixée.